

React *Hooks*

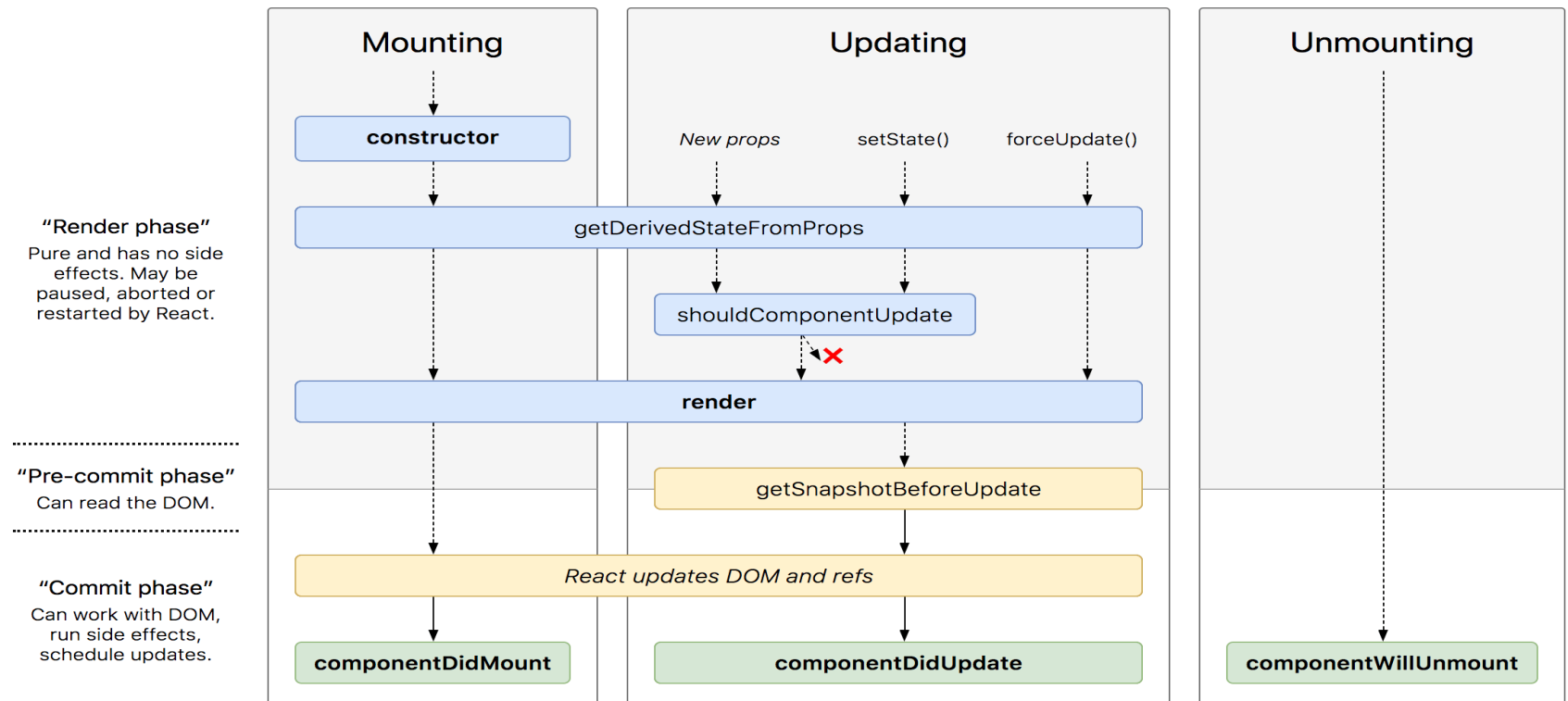
LEXICON

Objectives

- Component lifecycle
- React Hooks
- React **useState**
- React **useEffect**
- Form Validation
- React Hook Form
- Practice

Component Lifecycle

- The diagram below is from the [official React documentation](#) showcasing the different React lifecycle methods and when they are invoked.
- The `render()` is the most used lifecycle method.
- Lifecycle methods:
 - Initial
 - Mounting
 - Updating
 - Unmounting



React Hooks

- Hook is a new concept in React 16.8
- Are functions that allow developer “hook into” state and lifecycle from **function component**
- To reuse the stateful logic in functional components
- To Implement complex components with stateful logic
- Provide a simple way to control the component lifecycle
- Raise code readability & scalability

useState

- `useState()` is a new function that allows to have **state variables** in **function** components
- Returns a pair value: current state & a function that updates the current state
- Used to design complex UI
- Split one component into smaller
- Clean and Simple
- Reusable

Initial state

```
const [name, setName] = useState('');
```

Current State

Function that updates the current state

- ❖ It is hard to reuse stateful logic between components in **Traditional Class Component**

```
import { useState } from 'react';

export const Person = () => {
  const [name, setName] = useState('');

  const handleNameChange = (e) => {
    setName(e.target.value);
  };

  return (
    <div>
      <input type="text" value={name} onChange={handleNameChange} />
      <Greeting name={name} />
    </div>
  );
};

const Greeting = (props) => {
  const [message, setMessage] = useState('Welcome to my App');
  return (
    <div>
      <p> {message} <b> {props.name} </b> </p>
    </div>
  );
};
```

useEffect

- `useEffect()` is a Hook function
- Is used for **side-effects** (do something after render)
- Takes two arguments `useEffect(callback, [dependencies]);`
- `callback` is the function containing the side-effect logic
- `dependencies` are an optional array of dependencies (state, props and...)
- `useEffect` executes `callback` after every render and first render
- `useEffect` executes `callback` only if dependencies have changed between renderings

```
import { useEffect } from 'react';
```

useEffect

- `useEffect` executes `callback` after every render and first render
- `useEffect` executes `callback` only if dependencies have changed between renderings

```
import React, { useState, useEffect } from "react";
export const UseEffectExample = () => {
  const [showData, setShowData] = useState(false);
```

```
  const handleShowData = () => {
    console.log("showData executed!");
    setShowData(true);
  };
  const handleHideData = () => {
    console.log("hideData executed!");
    setShowData(false);
  };
  return (
    <div className="container">
```

```
      <h3>UseEffect Example</h3>
      <div className="row">
        <div className="col">
```

```
          <button type="button" className="btn btn-info" onClick={handleShowData}>Show</button>
          <button type="button" className="btn btn-warning" onClick={handleHideData}>Hide</button>
        </div>
      </div>{showData && <b>Test Message</b>}
    </div>
  ); }
```

callback function

```
useEffect(() => {
  console.log("useEffect executed!");
}, [showData]);
```

dependency

Form Validation

- Forms are an essential part of how users interact with applications
- Validation user's data through the form is an important responsibility for developer
- There are several ways to validate forms in React like **validator functions** or use **libraries**

```
const [email, setEmail] = useState("");
const [errors, setErrors] = useState({email: ''});

const validate = () => {
  let isValid = true;
  if (!email) {
    errors.email = 'Email address is required';
    setErrors({email: errors.email});
    isValid = false;
  } else if (!/\S+@\S+\.\S+/.test(email)) {
    errors.email = 'Email address is invalid';
    setErrors({email: errors.email});
    isValid = false;
  }
  return isValid;
}
```

Email address is required

```
const saveData = () => {
  const isValid = validate();
  if (isValid) {
    console.log("Email:", email);
    setErrors({email: ''})
  }
};
```

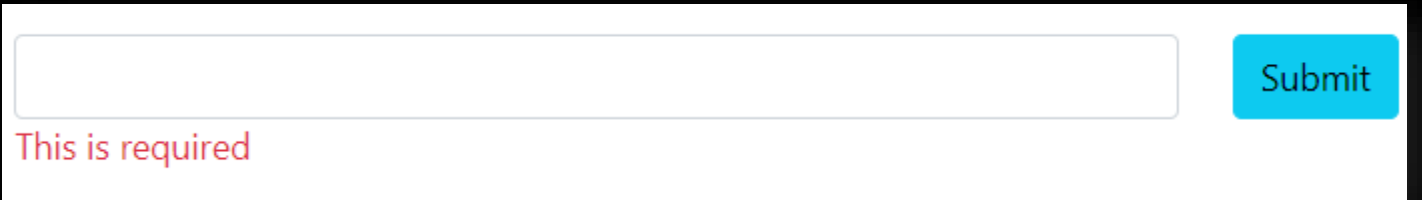

React Hook Form

- [React-hook-form](#) is a library helps developer to **validate forms** in a simple way
- Is a library without any other dependencies
- Is easy and straightforward to use
- To install React Hook Form: `npm install react-hook-form`
- To use: First, import the `useForm` Hook function from the `react-hook-form` package.

```
import { useForm } from "react-hook-form";
```

React Hook Form

```
import React, { useState } from "react";
import { useForm } from "react-hook-form";
export const ValidateForm = () => {
```



This is required

```
const { register, handleSubmit, formState: { errors } } = useForm();

const saveData = (data) => { console.log(data); };
return (
  <div className="container">
    <form className="form-control mt-3 p-3" onSubmit={handleSubmit(saveData)} >
      <div className="row">
        <div className="col">
          <input type="text" {...register("email", { required: true })} className="form-control" />
          {errors.email && <span className="text-danger">This is required</span>}
        </div>
        <div className="col">
          <button type="submit" className="btn btn-info"> Submit </button>
        </div>
      </div>
    </form>
  </div>
)
```

Practice

Project setup:

1. Create a new react app.
2. Import the react app in to the VSC.
3. Make sure to remove the default codes generated by React.
4. Create index.js file in the **src** folder + index.html in **public** folder.
5. Render a message in the index.js file.
6. Make sure to run the react application without errors.
7. Install Bootstrap using *npm install bootstrap*
8. Install React Hook Form using *npm install react-hook-form*
9. Commit & Push.

Defining components:

1. Create a js file called **Skill.js** using arrow function.
2. Define a **const** variable like **skills** using **useState** with initialState as an array.
3. Define a **ShowData** component that takes skills as props and returns `<ul>` with list of skills.
 - ❖ Skill object can have properties like: id, title
4. Define a **Form** component using **useForm** React Hook function.
 - ❖ Form component should have an **event for adding data to the list**.
 - ❖ When you add a new skill, you should create a random id for skill object.
5. Push when all components are done.

Fullstack Developer Skills

- React
- Java
- SQL



This is required

Links

<https://projects.wojtekmaj.pl/react-lifecycle-methods-diagram/>

<https://react-hook-form.com/>